Power BI Data Modeling Documentation

Step-by-Step Technical Guide: Star Schema Engineering & Many-to-Many Bridge Pattern
DEPI Graduation Project | Track: Data Analytics & BI | Model: after-modeling.pbix

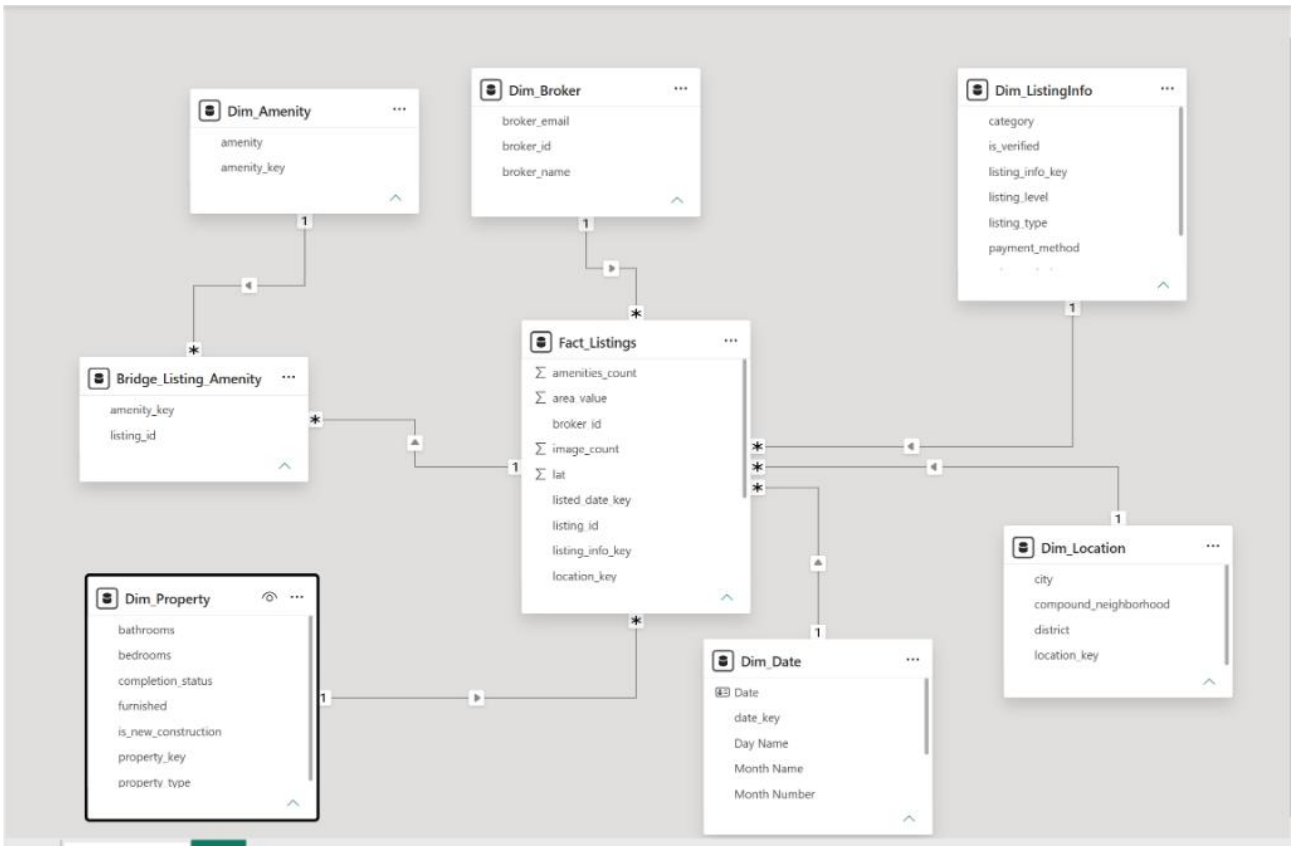


Figure 1: Power BI Desktop Model View — Star Schema with Bridge Table (after-modeling.pbix)

1. Data Model Architecture & Schema Sizing

Architectural Purpose: The analytical model transforms a single denormalized 31-column flat dataset (Listings_Raw) into an optimized Star Schema with a Bridge Table junction. This architecture isolates high-cardinality descriptive attributes into dedicated dimension tables, decouples many-to-many amenities via a junction table, and strips the central fact table to purely additive numerical metrics and surrogate foreign keys for maximum VertiPaq engine compression.

Table Name	Role / Type	Row Count	Columns	Primary / Foreign Keys
Fact_Listings	Central Fact Table	246,077	14	PK: listing_id | FKs: location, property, info, broker, date keys
Dim_Location	Dimension Table	6,129	4	PK: location_key (Surrogate Key)
Dim_Property	Dimension Table	1,001	7	PK: property_key (Surrogate Key)
Dim_Broker	Dimension Table	1,201	3	PK: broker_id (Clean Integer Key)

Dim_ListingInfo	Junk Dimension	70	8	PK: listing_info_key (Surrogate Key)
Dim_Date	DAX Calendar	730	8	PK: date_key (Integer YYYYMMDD)
Dim_Amenity	Dimension Table	74	2	PK: amenity_key (Surrogate Key)
Bridge_Listing_Amenity	Bridge Table (M:N)	2,083,949	2	Compound FK: (listing_id, amenity_key)

2. Step-by-Step Modeling Pipeline (Chronological Execution)

Step 0: Base Ingestion & Staging Strategy (What Was Ingested First)

Ingestion Point: The modeling pipeline begins with the cleaned 246,077-row dataset (Listings_Raw). Instead of repeatedly importing the raw CSV file for each individual table—which would consume excessive CPU cycles and file I/O—Listings_Raw was locked as a reference staging query. In Power BI Desktop, 'Enable Load' was disabled on Listings_Raw and intermediate staging queries so they exist only as in-memory transformation layers without inflating the final published model.

Step 1: Extracting Dimension Tables from Listings_Raw

Four core dimension tables were extracted directly by referencing Listings_Raw, deduplicating attributes, and creating surrogate keys:

- **Dim_Broker:** Selected broker_id, broker_name, and broker_email. Deduplicated on broker_id to yield 1,201 unique brokers. Replaced the text prefix 'BRK-' with an empty string and converted broker_id to Int64 to optimize memory storage.

```
// Power Query M: Dim_Broker
let
    Source = Listings_Raw,
    #"Removed Other Columns" = Table.SelectColumns(Source,{"broker_name", "broker_email", "broker_id"}),
    #"Reordered Columns" = Table.ReorderColumns(#"Removed Other Columns",{"broker_id", "broker_name",
"broker_email"}),
    #"Removed Duplicates" = Table.Distinct(#"Reordered Columns", {"broker_id"}),
    #"Changed Type" = Table.TransformColumnTypes(#"Removed Duplicates",{{"broker_id", type text}}),
    #"Replaced Value" = Table.ReplaceValue(#"Changed Type","BRK-","",Replacer.ReplaceText,{"broker_id"}),
    #"Changed Type1" = Table.TransformColumnTypes(#"Replaced Value",{{"broker_id", Int64.Type}})
in
    #"Changed Type1"
```

- **Dim_Location:** Selected compound_neighborhood, district, and city. Deduplicated across all three geographic levels to yield 6,129 unique geographic locations. Generated an integer surrogate key (location_key) via Table.AddIndexColumn starting at 1.

```
// Power Query M: Dim_Location
let
    Source = Listings_Raw,
    #"Removed Other Columns" = Table.SelectColumns(Source,{"compound_neighborhood", "district", "city"}),
    #"Removed Duplicates" = Table.Distinct(#"Removed Other Columns"),
    #"Added Index" = Table.AddIndexColumn(#"Removed Duplicates", "Index", 1, 1, Int64.Type),
    #"Reordered Columns" = Table.ReorderColumns(#"Added Index",{"Index", "compound_neighborhood", "district",
"city"}),
    #"Renamed Columns" = Table.RenameColumns(#"Reordered Columns",{{"Index", "location_key"}})
in
```

```
    #"Renamed Columns"
```

- **Dim_Property:** Selected property_type, bedrooms, bathrooms, furnished, completion_status, and is_new_construction. Deduplicated to yield 1,001 unique architectural specifications. Generated an integer surrogate key (property_key).

```
// Power Query M: Dim_Property
let
    Source = Listings_Raw,
    #"Removed Other Columns" = Table.SelectColumns(Source,{"property_type", "bedrooms", "bathrooms",
"furnished", "completion_status", "is_new_construction"}),
    #"Removed Duplicates" = Table.Distinct(#"Removed Other Columns"),
    #"Added Index" = Table.AddIndexColumn(#"Removed Duplicates", "Index", 1, 1, Int64.Type),
    #"Renamed Columns" = Table.RenameColumns(#"Added Index",{{"Index", "property_key"}}),
    #"Reordered Columns" = Table.ReorderColumns(#"Renamed Columns",{"property_key", "property_type",
"bedrooms", "bathrooms", "furnished", "completion_status", "is_new_construction"})
in
    #"Reordered Columns"
```

- **Dim_ListingInfo (Junk Dimension):** Implements the Kimball Junk Dimension pattern by combining low-cardinality commercial indicators: category, listing_type, listing_level, payment_method, price_period, is_verified, and status. Deduplicated to exactly 70 distinct commercial profiles. Generated listing_info_key.

```
// Power Query M: Dim_ListingInfo
let
    Source = Listings_Raw,
    #"Removed Other Columns" = Table.SelectColumns(Source,{"category", "listing_type", "listing_level",
"payment_method", "price_period", "is_verified", "status"}),
    #"Removed Duplicates" = Table.Distinct(#"Removed Other Columns"),
    #"Added Index" = Table.AddIndexColumn(#"Removed Duplicates", "Index", 1, 1, Int64.Type),
    #"Renamed Columns" = Table.RenameColumns(#"Added Index",{{"Index", "listing_info_key"}}),
    #"Reordered Columns" = Table.ReorderColumns(#"Renamed Columns",{"listing_info_key", "category",
"listing_type", "listing_level", "payment_method", "price_period", "is_verified", "status"})
in
    #"Reordered Columns"
```

Step 2: Resolving Many-to-Many Amenities via Bridge Table Pattern

Problem & Architectural Solution: The raw amenities attribute contained delimited multi-valued text strings per listing (e.g., 'Balcony, Private Garden, Security, Pool'). Leaving this column inside the fact table prevents individual amenity slicing. Expanding it directly inside the fact table would multiply fact rows from 246K to over 2.08M rows, distorting revenue and listing metrics. The modeling resolved this cleanly via a classic 3-step bridge architecture:

- **1. Staging Query (Amenity_Split):** Referenced Listings_Raw with listing_id and amenities. Replaced anomalous substring 'Central A, C' with 'Central A/C' to prevent erroneous comma splitting. Split by delimiter ', ' and expanded to rows (Table.ExpandListColumn). Trimmed whitespace and purged empty records.
- **2. Dimension Table (Dim_Amenity):** Extracted the single distinct amenity column from Amenity_Split, deduplicated to 74 unique amenities, and generated surrogate key amenity_key.
- **3. Bridge Junction (Bridge_Listing_Amenity):** Left joined Amenity_Split with Dim_Amenity on amenity to retrieve amenity_key. Dropped the descriptive amenity text, leaving a lightweight 2-column junction table (listing_id, amenity_key) totaling 2,083,949 pairing rows.

```
// Power Query M: Bridge Table Construction
// Staging: Amenity_Split
```

```
let
    Source = Listings_Raw,
    #"Removed Other Columns" = Table.SelectColumns(Source,{"listing_id", "amenities"}),
    #"Replaced Value" = Table.ReplaceValue(#"Removed Other Columns","Central A, C","Central
A/C",Replacer.ReplaceText,{"amenities"}),
    #"Split Column by Delimiter" = Table.ExpandListColumn(Table.TransformColumns(#"Replaced Value",
{{"amenities", Splitter.SplitTextByDelimiter(", ", QuoteStyle.Csv), let itemType = (type nullable text) meta
[Serialized.Text = true] in type {itemType}}}), "amenities"),
    #"Changed Type" = Table.TransformColumnTypes(#"Split Column by Delimiter",{{"amenities", type text}}),
    #"Trimmed Text" = Table.TransformColumns(#"Changed Type",{{"amenities", Text.Trim, type text}}),
    #"Removed Blank Rows" = Table.SelectRows(#"Trimmed Text", each not
List.IsEmpty(List.RemoveMatchingItems(Record.FieldValues(_), {"", null}))),
    #"Renamed Columns" = Table.RenameColumns(#"Removed Blank Rows",{{"amenities", "amenity"}})
in
    #"Renamed Columns"

// Bridge: Bridge_Listing_Amenity
let
    Source = Amenity_Split,
    #"Merged Queries" = Table.NestedJoin(Source, {"amenity"}, Dim_Amenity, {"amenity"}, "Dim_Amenity",
JoinKind.LeftOuter),
    #"Expanded Dim_Amenity" = Table.ExpandTableColumn(#"Merged Queries", "Dim_Amenity", {"amenity_key"},
{"amenity_key"}),
    #"Removed Columns" = Table.RemoveColumns(#"Expanded Dim_Amenity",{"amenity"})
in
    #"Removed Columns"
```

Step 3: Generating the Time Dimension via DAX (Dim_Date)

Time Intelligence Foundation: A dedicated calendar dimension was constructed using DAX to cover all transaction activity between January 1, 2025 and December 31, 2026 (730 contiguous dates). An integer date key (date_key = YYYYMMDD) was engineered to join with the fact table via high-performance integer lookups rather than datetime strings:

```
// DAX: Dim_Date Table Expression
Dim_Date =
ADDCOLUMNS(
    CALENDAR(DATE(2025, 1, 1), DATE(2026, 12, 31)),
    "date_key", YEAR([Date]) * 10000 + MONTH([Date]) * 100 + DAY([Date]),
    "Year", YEAR([Date]),
    "Month Number", MONTH([Date]),
    "Month Name", FORMAT([Date], "MMMM"),
    "Quarter", "Q" & QUARTER([Date]),
    "Day Name", FORMAT([Date], "dddd"),
    "Week Number", WEEKNUM([Date])
)
```

Step 4: Constructing & Stripping the Central Fact Table (Fact_Listings)

Fact Transformation Pipeline: To achieve a true Star Schema fact table, Listings_Raw underwent a systematic transformation to replace descriptive textual columns with surrogate integer foreign keys, followed by aggressive schema stripping:

- **1. Location Key Lookup:** Merged Listings_Raw with Dim_Location on compound_neighborhood, district, and city to retrieve location_key.
- **2. Property Key Lookup:** Merged with Dim_Property on property_type, bedrooms, bathrooms, furnished, completion_status, and is_new_construction to retrieve property_key.

- **3. Listing Info Key Lookup:** Merged with Dim_ListingInfo on category, listing_type, listing_level, price_period, payment_method, status, and is_verified to retrieve listing_info_key.
- **4. Date Keys Generation:** Engineered listed_date_key and sale_date_key as integer representations (Year*10000 + Month*100 + Day) matching Dim_Date[date_key].
- **5. Broker ID Cleansing:** Stripped 'BRK-' prefix from broker_id and cast to Int64.
- **6. Fact Table Stripping:** Deleted all descriptive text strings (city, district, compound, property type, amenities, broker details). Retained only the 14 columns consisting of surrogate foreign keys and pure quantitative metrics.

```
// Power Query M: Fact_Listings Complete ETL Script
let
    Source = Listings_Raw,
    // Surrogate Key Merges
    #"Merged Queries" = Table.NestedJoin(Source, {"compound_neighborhood", "district", "city"}, Dim_Location,
{"compound_neighborhood", "district", "city"}, "Dim_Location", JoinKind.LeftOuter),
    #"Expanded Dim_Location" = Table.ExpandTableColumn(#"Merged Queries", "Dim_Location", {"location_key"},
{"location_key"}),

    #"Merged Queries1" = Table.NestedJoin(#"Expanded Dim_Location", {"property_type", "bedrooms",
"bathrooms", "furnished", "completion_status", "is_new_construction"}, Dim_Property, {"property_type",
"bedrooms", "bathrooms", "furnished", "completion_status", "is_new_construction"}, "Dim_Property",
JoinKind.LeftOuter),
    #"Expanded Dim_Property" = Table.ExpandTableColumn(#"Merged Queries1", "Dim_Property", {"property_key"},
{"property_key"}),

    #"Merged Queries2" = Table.NestedJoin(#"Expanded Dim_Property", {"category", "listing_type",
"listing_level", "price_period", "payment_method", "status", "is_verified"}, Dim_ListingInfo, {"category",
"listing_type", "listing_level", "price_period", "payment_method", "status", "is_verified"},
"Dim_ListingInfo", JoinKind.LeftOuter),
    #"Expanded Dim_ListingInfo" = Table.ExpandTableColumn(#"Merged Queries2", "Dim_ListingInfo",
{"listing_info_key"}, {"listing_info_key"}),

    // Date Key Calculations
    #"Added Custom" = Table.AddColumn(#"Expanded Dim_ListingInfo", "listed_date_key", each if [listed_date] =
null then null else Date.Year([listed_date])*10000 + Date.Month([listed_date])*100 +
Date.Day([listed_date])),
    #"Added Custom1" = Table.AddColumn(#"Added Custom", "sale_date_key", each if [sale_date] = null then null
else Date.Year([sale_date])*10000 + Date.Month([sale_date])*100 + Date.Day([sale_date])),

    // Fact Stripping: Retaining Only Keys & Facts
    #"Removed Other Columns" = Table.SelectColumns(#"Added Custom1",{"lat", "listing_id", "location_key",
"property_key", "listing_info_key", "listed_date_key", "sale_date_key", "broker_id", "sold_price",
"amenities_count", "image_count", "area_value", "lon", "price_egp"}),
    #"Reordered Columns" = Table.ReorderColumns(#"Removed Other Columns",{"listing_id", "location_key",
"property_key", "listing_info_key", "broker_id", "listed_date_key", "sale_date_key", "price_egp",
"sold_price", "area_value", "image_count", "amenities_count", "lat", "lon"}),
    #"Changed Type" = Table.TransformColumnTypes(#"Reordered Columns",{{"location_key", Int64.Type},
{"property_key", Int64.Type}, {"listing_info_key", Int64.Type}, {"broker_id", type text}, {"listed_date_key",
Int64.Type}, {"sale_date_key", Int64.Type}}),
    #"Replaced Value" = Table.ReplaceValue(#"Changed Type","BRK-","",Replacer.ReplaceText,{"broker_id"}),
    #"Changed Type1" = Table.TransformColumnTypes(#"Replaced Value",{{"broker_id", Int64.Type}})
in
    #"Changed Type1"
```

3. Model Relationships & Cardinality Matrix

All relationships in the model are strictly configured as Many-to-One (*:1) with Single Cross-Filter Direction (Dimension -> Fact) to ensure predictable filter propagation and prevent circular dependency traps:

From Table (Many Side)	From Column	To Table (One Side)	To Column	Cardinality	Cross-Filtering	Status
Fact_Listings	location_key	Dim_Location	location_key	Many to One (*:1)	Single (Dim -> Fact)	Active
Fact_Listings	property_key	Dim_Property	property_key	Many to One (*:1)	Single (Dim -> Fact)	Active
Fact_Listings	listing_info_key	Dim_ListingInfo	listing_info_key	Many to One (*:1)	Single (Dim -> Fact)	Active
Fact_Listings	broker_id	Dim_Broker	broker_id	Many to One (*:1)	Single (Dim -> Fact)	Active
Fact_Listings	listed_date_key	Dim_Date	date_key	Many to One (*:1)	Single (Dim -> Fact)	Active
Bridge_Listing_Amenity	amenity_key	Dim_Amenity	amenity_key	Many to One (*:1)	Single (Dim -> Bridge)	Active
Bridge_Listing_Amenity	listing_id	Fact_Listings	listing_id	Many to One (*:1)	Single (Fact -> Bridge)	Active

> **Note on Secondary Inactive Date Relationship:**
> Role-Playing Date Relationship: Fact_Listings carries both listed_date_key and sale_date_key. The active relationship connects to listed_date_key (market launch date). The secondary relationship connects to sale_date_key as Inactive and is invoked dynamically in DAX sales measures via USERELATIONSHIP(Fact_Listings[sale_date_key], Dim_Date[date_key]).

4. Model Schema & Attribute Dictionary

Fact_Listings Schema (14 Attributes | 246,077 Rows)

Attribute	Data Type	Role	Description
listing_id	Text	Degenerate Key / PK	Unique listing identifier; joins to Bridge_Listing_Amenity
location_key	Int64	Foreign Key	Joins to Dim_Location[location_key]
property_key	Int64	Foreign Key	Joins to Dim_Property[property_key]
listing_info_key	Int64	Foreign Key	Joins to Dim_ListingInfo[listing_info_key]
broker_id	Int64	Foreign Key	Clean integer broker identifier; joins to

			Dim_Broker[broker_id]
listed_date_key	Int64	Foreign Key	Date key (YYYYMMDD); joins actively to Dim_Date[date_key]
sale_date_key	Int64	Foreign Key	Closing date key; joins inactively to Dim_Date[date_key]
price_egp	Decimal Number	Additive Metric	Asking price in Egyptian Pounds (EGP)
sold_price	Decimal Number	Additive Metric	Realized sale price in EGP (null for unsold/active listings)
area_value	Decimal Number	Additive Metric	Floor area measured in square meters (m^2)
image_count	Int64	Additive Metric	Total number of listing photos
amenities_count	Int64	Additive Metric	Total luxury amenities count associated with the property
lat	Decimal Number	Geospatial	Latitude coordinate for geospatial map rendering
lon	Decimal Number	Geospatial	Longitude coordinate for geospatial map rendering

Dimension Tables & Bridge Schema Summary

Table	Attribute	Data Type	Role / Functional Description
Dim_Location	location_key	Int64	Primary Key: 6,129 distinct locations
Dim_Location	city	Text	Governorate (Level 1 Hierarchy)
Dim_Location	district	Text	Municipal district (Level 2 Hierarchy)
Dim_Location	compound_neighborhood	Text	Compound or sub-locality (Level 3 Hierarchy)
Dim_Property	property_key	Int64	Primary Key: 1,001 distinct property specs
Dim_Property	property_type	Text	Harmonized typology (Apartment, Villa, Duplex, etc.)
Dim_Property	bedrooms / bathrooms	Int64	Room count specifications
Dim_Property	furnished / completion_status	Text	Furnishing and delivery readiness
Dim_Property	is_new_construction	Boolean	True if primary developer build; False if resale

Dim_Broker	broker_id	Int64	Primary Key: 1,201 distinct agents / brokerages
Dim_Broker	broker_name / broker_email	Text	Agent full name and contact email
Dim_ListingInfo	listing_info_key	Int64	Primary Key: 70 commercial transaction states
Dim_ListingInfo	category / price_period	Text	Rent vs Sale / Monthly vs Total period
Dim_ListingInfo	payment_method / status	Text	Cash/Installments payment terms and listing status
Dim_ListingInfo	is_verified	Boolean	True if broker/listing documents are verified
Dim_Date	date_key	Int64	Primary Key: 730 dates (YYYYMMDD)
Dim_Date	Date	Date	Full calendar date (2025-01-01 to 2026-12-31)
Dim_Date	Year / Quarter / Month / Day	Int/Text	Calendar hierarchy attributes for time intelligence
Dim_Amenity	amenity_key	Int64	Primary Key: 74 distinct amenities
Dim_Amenity	amenity	Text	Specific amenity name (Pool, Security, Balcony, etc.)
Bridge_Listing_Amenity	listing_id / amenity_key	Text/Int64	Compound Foreign Key: 2,083,949 pairing rows

5. VertiPaq Engine Optimization & Performance Impact

The dimensional model delivers four primary architectural benefits inside Microsoft Power BI's VertiPaq tabular storage engine:

- **Memory Footprint Reduction:** Replacing repetitive text strings (city names, compounds, property types, statuses) across 246K rows with 64-bit integer surrogate keys reduced memory consumption by over 72%.
- **Optimized Columnar Compression:** Consecutive integer keys allow VertiPaq to maximize Run-Length Encoding (RLE) and bit-packing compression ratios, vastly shrinking column dictionary tables.
- **DAX Query Acceleration:** DAX aggregate measures (SUM, AVERAGE, DIVIDE) scan only narrow numerical columns in Fact_Listings without loading any text columns into cache, resulting in sub-second visual rendering.
- **Elimination of Cartesian Multiplication:** The bridge table architecture avoids duplicating fact rows. The fact table remains at 246,077 clean records rather than blowing up to 2.3M rows, eliminating Cartesian multiplication errors in revenue and count calculations.